

中文预训练语言模型的句法几何：结构探针、预训练策略与控制检验

The Geometry of Chinese Syntax in Pre-trained Language Models: Structural Probes, Pre-training Objectives, and Control Tasks

状态：定稿 v6 (2026-10-04)。逐章定稿完成；全部数字来自 results/*.json (四模型 × 13 层主线探针、三模型均值池化消融、14 组控制实验、四模型 MDL)，由 src/summarize.py 自动汇总为 docs/results-tables.md；图由 src/plot_results.py 生成于 figures/ (正文 8 张表共 615 个单元格已与结果文件逐一复核，中英两份逐行对应、数字 token 完全一致)。参考文献书目经一手页面核实 (首轮 2026-10-02, RoBERTa 一条 2026-10-04 补核)。第 5 节列出的局限是作者已知且保留的项目。

摘要

中文预训练语言模型 (PLM) 的内部表示是否编码了句法结构？本文将 Hewitt & Manning (2019) 的结构探针方法系统应用于四个同规模模型——三个预训练策略不同的中文单语模型 (bert-base-chinese、RoBERTa-wwm-ext、MacBERT) 与多语 mBERT——在 UD Chinese-GSDSimp 树库上逐层检验依存句法的几何可解码性，并施加两个方向的控制任务 (selectivity) 与 MDL 信息论检验。我们发现：(1) 汉语句法树可被线性变换后的平方距离有效重建，三个单语模型的 dev UUAS 峰值**同在第 8 层** (0.587、0.587、0.589)，test 峰值 0.596–0.605，远高于随机树基线 0.096；曲线呈先升后降的驼峰形。(2) 三种遮蔽策略 (原版 MLM、全词遮蔽、纠错式遮蔽) 收敛到几乎相同的句法几何：峰值层一致，全层最大差距 2.6 个百分点，峰值层仅 0.2 个百分点；把词表示从“词首子词”换成“词内均值”后绝对值整体上移约 3 个百分点、深度与距离两类 selectivity 同步上移 (层 8 距离 Δ UUAS 0.49 $\rightarrow$ 0.52)，而峰值层与曲线形状不变——结论不依赖中文特有的字 $\leftrightarrow$ 词对齐方式。(3) 多语 mBERT 在同一条流水线下并不弱于单语中文模型 (dev 峰值 0.604、test 0.619，均高于三个单语模型)，但 UUAS 峰值提前到第 7 层，距离 selectivity 与 MDL 最低码长 (158.3 kbit) 也同落在层 7 (三条线方向一致，各自差距均在 1 个百分点量级)；它的深度 selectivity (第 8 层 +0.095) 落在单语模型的区间内，说明这一结果不是词形记忆造成的假象。(4) 控制检验显示，深度探针在低层 (0–4 层) 的 selectivity 为负 (低至 -0.16) ——探针在那里主要靠词形/词型记忆而非句法；selectivity 的转正与峰值恰好与句法峰重合于第 8 层 (Δ = 0.07、0.07、0.11)。距离探针在固定随机树控制下全层 selectivity 强正 (Δ UUAS 0.28 $\rightarrow$ 0.49)，控制探针的 dev 相关 $\approx$ 0 而 train 相关为正，说明随机结构只是被“背”下来、并未进入表示。(5) MDL 检验独立复现同一结论：45 类依存关系标签的在线编码长度在第 8 层最低 (157–161 kbit，压缩率 3.37–3.45 $\times$ ，test 准确率 0.798–0.801)。三条互相独立的证据线 (探针指标、控制任务、信息论) 指向同一层。据我们所知，这是首次对中文 PLM 家族进行带完整控制检验的结构探针研究。

关键词：结构探针；可解释性；中文 BERT；依存句法；控制任务；最小描述长度

1 引言

预训练语言模型在各种句法敏感任务上表现出色，但“下游分数高”并不等于“内部真的学会了语言学意义上的结构”。区分二者需要直接检查表示本身：如果某个语言性质真的以某种几何形式存在于表示中，那么一个足够简单 (线性) 的读出函数应当能从表示中把它恢复出来。

中文在这条问题线上有独立价值。汉语没有形态变化、以字为基本书写单位 (对当前一代模型而言也基本以字为 token)、语序承担主要语法功能；英文上得到的“句法信息集中在中层”一类结论 (如 Tenney et al. 2019 的层序分析) 不能自动搬过来。已有跨语言证据 (Chi, Hewitt & Manning 2020) 甚至把中文列为 mBERT 编码较弱的一档。另一方面，中文 PLM 家族提供了一个罕见的干净对照组：bert-base-chinese、RoBERTa-wwm-ext (HFL 按 RoBERTa (Liu et al. 2019) 的配方在中文上重训、改用全词遮蔽 wwm)、MacBERT (遮蔽时用近义词替换，纠错式目标) 三者的**规模、层数、架构完全相同**，差别只在预训练目标。这是研究“训练目标 $\rightarrow$ 内部结构”因果链条的天然实验。

本文提出三个研究问题：

- **RQ1 (存在与位置)**：各层表示能否经线性变换重建 UD 汉语依存树？峰值在哪一层？
- **RQ2 (预训练策略与模型来源)**：改变遮蔽策略 (MLM / wwm / 纠错式) 或语言覆盖范围 (多语 mBERT)，改变的是句法编码的几何本身，还是仅仅下游分数？
- **RQ3 (检验下存活)**：上述发现在 selectivity 控制与 MDL 检验下是否仍然成立？

贡献：

1. 首次 (据我们所知) 把"结构探针 + 中文单语模型家族对照 + 双向 selectivity 控制 + MDL"的组合做到中文上；最近的相关工作是 Zheng & Liu (2023)，但它是单模型、注意力头/表层任务、无结构探针与无控制。
2. 发现三个模型的中层句法几何高度趋同 (峰值同层、全层强度差 ≤ 2.6 个百分点，峰值层差 0.2 个百分点)，提示句法几何对预训练遮蔽策略鲁棒；把词表示从"词首子词"换成"词内均值"后峰值层与曲线形状不变、两类 selectivity 同步上移，说明结论也不依赖中文特有的字↔词对齐方式。
3. 多语 mBERT 在同一条流水线下并不弱于单语中文模型 (dev 峰值 0.604、test 0.619)，但 UUAS 峰值、距离 selectivity 与 MDL 最低码长三条线一致地把峰值提前到层 7——这与"中文在 mBERT 内部编码偏弱"的跨语言结论并不冲突，二者问的是不同层面的问题 (4.2b、第 5 节)。
4. 用控制任务定位了"句法峰"的可信区间：深度控制的 selectivity 在层 0–4 为负 (低层的表面可解码性主要来自词形记忆)，层 5 起转正；只有第 5 层以上、尤其第 8 层的信号在两类控制下都站得住。
5. MDL 用比特数这把完全不同的尺子独立复现第 8 层；在线编码 (每块只用前缀数据训练) 让"背下整张表"无法获益，因此"探针容量过强造成假阳性"这一质疑不成立。
6. 开源可复现的极简流水线：单台笔记本 CPU (无 GPU) 约 30 小时跑完全部结果 (主线探针 10.5 h、距离控制 15.1 h、深度控制 1.4 h、MDL 0.3 h、表示抽取约 2.5 h)；代码、全部结果文件与论文源码见 <https://github.com/Shizuku-keop/chinese-plm-syntax-probe>。

2 相关工作

结构探针谱系。Hewitt & Manning (2019) 提出距离探针：学线性变换 B ，令变换后向量的平方距离逼近依存树距离，再用最小生成树重建句法树，用 UUAS 评估 (按惯例不计标点——本文的评估包含标点，见 3.1 与第 5 节局限⑥)。他们在英文 Penn Treebank 上对 ELMo 与 BERT-base/large 逐层验证了这种变换确实存在 (BERT-large > BERT-base > ELMo)，并发现所需的线性变换秩出奇地低 (k 超过 64–128 后收益不再增加)；案例分析用的是 BERT-large 的第 16 层 (共 24 层) 与 ELMo 的第 1 层。Tenney et al. (2019) 用 edge probing 发现各层大致按经典 NLP 流水线顺序处理任务；Clark et al. (2019) 定位到跟踪依存关系的注意力头 (上述层间分工的证据与争议见 Rogers et al. 2020 的综述)。Kulmizev et al. (2020) 在 13 种语言上比较 UD 与表层句法标注体系 (SUD)，发现 BERT 与 ELMo 都一致偏好 UD，且偏好强度随语言与层变化——说明"探针测到的结构"与所选形式化体系有关，这也是本文固定单一树库与标注体系、并把该点列入局限的原因。

探针的方法论批判。Hewitt & Liang (2019) 指出高准确率可能来自探针记住了词形—标签的任意映射，因此提出控制任务与 $\text{selectivity} = \text{语言任务指标} - \text{控制任务指标}$ 。Voita & Titov (2020) 与 Pimentel et al. (2020) 主张用信息量 (比特) 而非准确率衡量表示，把探针本身的复杂度计入代价 (MDL)。本文对这两类批判各做了回应：控制任务 (3.2) 与 MDL (3.3)。Pimentel et al. (2020) 提出的帕累托前沿扫描我们**没有做**——本文的探针容量固定 (rank 128 硬编码，见 3.5.10)，复杂度一侧改由 MDL 的编码长度来体现。

中文相关工作。CLiMP (Xiang et al. 2021)、SLING (Song et al. 2022)、ZhoBLiMP (Liu et al. 2026) 是行为基准 (看模型对句对概率的判断)，不探内部表示；Zheng & Liu (2023) 在 bert-base-chinese 上做了注意力头探针与表层任务探针，覆盖把/被/了/着/过，但没有结构探针、没有控制、只研究单一模型；Chi et al. (2020) 对 mBERT 在 11 个 UD 树库 (含 Chinese-GSD) 做结构探针，中文是编码较弱的一档，但同样没有控制检验。我们的位置：把"结构探针 + 模型家族对照 + 完整控制"第一次做到中文单语模型上。

3 方法

3.1 结构探针

距离探针。设第 l 层第 i 个词的表示为 $h_i^{(l)} \in \mathbb{R}^{768}$ ，学一个线性变换 $B \in \mathbb{R}^{128 \times 768}$ ，使

$$d_B(i, j) = \|B(h_i - h_j)\|^2$$

逼近树库中的依存树距离 $d_T(i, j)$ （两词在依存树上的无向路径长度）。损失为 $L = \sum_s \sum_{i < j} |d_T(i, j) - d_B(i, j)|$ （L1，对离群更稳健），用 Adam + CosineAnnealing 优化。注意这是目标函数的形式；实现上按**批内有效词对取均值**（`(pred-gold).abs()[pair_mask].mean()`），因此不同批的实际步长尺度由批内元素数决定，详见 3.5.4。

深度探针。同一形式的 $\|Bh_i\|^2$ 对齐节点深度（到根的距离），用于检验层级结构的可解码性。

评估。由预测距离矩阵取最小生成树（MST）得到无向依存边集合，报告 UUAS（无向边准确率，即预测边集合与 gold 无向边集合的交集比例）；另报告预测距离与 gold 距离的 Spearman 相关、深度 Spearman 与根节点准确率。需要提醒的是评估口径：本文实现**把标点词作为普通节点留在图里**（标点参与 MST、其依存边也计入 gold 集合），只排除根的父边（根没有父边），详见 3.5.4——这一点与“排除标点”的常见做法不同，会使 UUAS 的绝对值偏高，故本文数字只适合在本文流水线内部纵向比较。UUAS 是“能不能重建出一棵树”的硬指标，Spearman 是“距离矩阵整体是否单调一致”的软指标，两者互补。

数学注记。展开可得

$$\|B(h_i - h_j)\|^2 = (h_i - h_j)^\top M (h_i - h_j), \quad M = B^\top B \succeq 0, \text{rank}(M) \leq 128.$$

也就是说，距离探针在学一个**低秩马氏距离**（Mahalanobis distance）：M 半正定保证对称性与三角不等式——秩亏时 d_B 只是**伪度量**（ $d_B(i, j) = 0$ 不蕴含 $h_i = h_j$ ），满秩时才严格满足度量公理；秩约束则等价于一个假设——句法相关信息集中在一个不超过 128 维的子空间里（相对 768 维表示空间）。若把 M 写成谱分解 $M = \sum_k \lambda_k u_k u_k^\top$ ，则 $\|B(h_i - h_j)\|^2$ 是“把表示差投影到若干方向、再按特征值加权平方”的叠加；探针学到的 u_k 就是它认为承载句法区分度的方向。UUAS 之所以比 Spearman 关键，是因为它把“几何拟合得好”翻译成“能否从几何中重建离散的树”——这是可解码性主张的最小形式。

3.2 控制任务与 selectivity

深度控制。把每个词型（word type）随机指派一个“假深度”并在该词型的所有出现上保持固定，让同一个深度探针去拟合这个随机目标。这样的目标只能靠词型身份（词形/词表信息）猜，不能靠句法计算。selectivity 定义为 gold 深度 Spearman – 控制深度 Spearman；负值说明探针在该层主要在做词形记忆。

距离控制。对每个句子用 Prüfer 序列生成一棵与真树同长、但结构随机的树，让距离探针去拟合它。随机树与真树的区别在于：真树的距离由句法结构决定，随机树的距离几乎只能靠逐词记忆。若探针能“背下”训练句的随机树，它在 dev 上应当失败（因为 dev 是全新的随机树）。故事也是如此，见 4.3。

3.3 MDL 检验

准确率指标有一个内生的漏洞：容量更大的探针总能刷出更高分，分数高可能只说明探针强。MDL（最小描述长度）换一把尺子，直接数“把标签传过去需要多少比特”：如果一种表示真的把标签信息“摆在表面”，一个简单探针就能以很短的编码传递标签；反之，表示里没有该信息时，编码长度逼近均匀编码。探针本身的代价由在线编码方式体现——每块标签只能用前缀数据训练出的探针来编码，因此多出来的容量换不来更短的码长。

我们采用 Voita & Titov (2020) 的在线编码长度框架。把 train 全部词按固定随机序排列，切成前缀块（10%、20%、...、100%）。第一块没有任何可用信息，用均匀码编码，代价为 $n_1 \log_2 K$ （ $K = 45$ 为依存关系标签类别数）。对第 t 块，先用前 $t-1$ 块数据训练一个线性分类器（ $\text{dim} \rightarrow 45$ ，交叉熵 + Adam，每前缀固定 5 个 epoch），再用它输出的概率给本块金标签编码，代价为

$$L = n_1 \log_2 K + \sum_{t \geq 2} \sum_{i \in \text{块}t} -\log_2 p_\theta(\text{gold}_i | h_i).$$

L 越小，表示越"好"；报告压缩率 = $n \log_2 K / L$ （均匀码长与实际码长之比）以及最终探针的 test 准确率作参考。要点是：码长在 train 上计算，且探针每块只能看到前缀数据，因此"背下整张表"不会带来收益。

3.4 实验设置

- **数据**：UD Chinese-GSDSimp（简体），train 3,997 / dev 500 / test 500 句；依存树解析自 CoNLL-U，含 45 类依存关系标签，CC BY-SA 4.0；树库随 UD 数据发布（框架引 Nivre et al. 2016，本身无独立论文）。
- **模型**：四个规模相同的模型（~102M 参数、12 层 Transformer，加输入层共 13 个抽取点）：中文单语 bert-base-chinese (Devlin et al. 2019)、hfl/chinese-roberta-wwm-ext (Cui et al. 2021)、hfl/chinese-macbert-base (Cui et al. 2020)，以及多语 mBERT (bert-base-multilingual-cased，亦出自 Devlin et al. 2019)。全部从本地目录加载，**冻结权重**，只读表示。
- **中文特有处理与消融**：UD 按词标注、而中文 BERT 按字切分，故需对齐；主实验取每个词的首字子词表示（--pooling first，对齐失败 0 例）。为检验这一选择的影响，另做一组**均值池化消融**（词内所有子词取均值，--pooling mean）：对三个单语模型重跑主线探针、深度控制、距离控制与 MDL。
- **超参**：距离/深度探针 rank = 128、epochs = 20、lr = 1e-3、batch_size = 32、seed = 0；MDL 探针 lr = 1e-3、每前缀 5 epochs、batch_size = 1024。
- **算力与耗时**：单台笔记本 CPU（无 GPU）。主线探针每模型约 88 分钟（13 层），深度控制每模型约 12 分钟，距离控制每模型约 80 分钟，MDL 每模型约 3 分钟；表示抽取每个"模型 × 池化"组合约 20–28 分钟，七个组合合计约 2.5 小时。按 results/*.json 的 seconds 字段累加，本仓库全部结果合计约 30 小时（主线探针 10.5 h、距离控制 15.1 h、深度控制 1.4 h、MDL 0.3 h，另加表示抽取约 2.5 h）。
- **验收门（跑通性自检）**：①解析句数 3,997 / 500 / 500；②字↔词对齐失配 0（"无子词覆盖回退"在个别词上出现 1 例，见 3.5.3）；③过拟合门 --overfit64：在 train 前 64 句上同时训练与评估距离探针，要求距离 Spearman > 0.9——层 7 实测 0.9455，OVERFIT GATE: PASS（日志 results/overfit64_bert.log）。这道门验证的是优化流程确实能拟合给定信号，与表示质量无关；④UUAS 落在合理区间（参照 results/baselines.json：随机树 0.096、线性链 0.426），此项为人工核对，代码中没有自动断言。全部验收门的清单与形式见 3.5.9。

3.5 程序实现

本节交代支撑 3.1–3.4 节全部结果的代码实现。全部脚本位于 src/（13 个 Python 文件，含一个只做前向冒烟的 smoke_test.py），另有四个批量驱动 shell 脚本；所有命令均以仓库根目录为工作目录运行，输入输出路径在代码中硬编码为相对路径（如 DATA_DIR = Path("data/ud_gsdsimp")、OUT_DIR = Path("results/representations"）；除 download_model.py 显式访问 ModelScope 外，其余脚本不联网（模型与树库需先落到本地）。

3.5.1 模块地图与数据流

完整链条是 "CoNLL-U → 表示张量 → 探针指标 → 表与图"，各阶段彼此只通过磁盘上的文件耦合（treebank.py 是唯一被多处 import 的解析库）：

阶段	脚本	输入	产物
树库解析 (库)	src/treebank.py	data/ud_gsdsimp/zh_gsdsimp-ud-{train,dev,test}.conllu	内存中的句列表（不落盘）
基线	src/baselines.py	三个 conllu	results/baselines.json
模型下载	src/download_model.py <id>	ModelScope 直链	models/<名称>/{config.json,vocab.txt,tokenizer*,model.safetensors}

阶段	脚本	输入	产物
表示抽取	src/extract.py models/<名称> [--pooling first\ mean]	conllu + 本地模型	results/representations/<名称>/{train,dev,test}.npz
主线探针	src/run_first_probe.py --model <名称>	conllu + 三个 npz	results/first_probe_<名称>.json
深度控制	src/run_control.py --control depth --model <名称>	同上	results/control_depth_<名称>.json
距离控制	src/run_control.py --control distance --model <名称>	同上	results/control_distance_<名称>.json
MDL	src/run_md1.py --model <名称>	conllu + npz	results/md1_<名称>.json
汇总表	src/summarize.py	上述全部 JSON	docs/results-tables.md
绘图	src/plot_results.py	上述全部 JSON	figures/fig1_syntax_geometry.png ... fig4_pooling_ablation.png

关键连接点：extract.py 只从树库取 words 与 # text（对齐用），距离/深度/依存标签一律由 treebank.parse_treebank 现算；探针脚本同时打开 conllu 与 npz，并要求两者**句序、词序逐句一致**——这一前提由词数断言兜住（3.5.9）。均值池化不引入新脚本，extract.py 把输出目录名写作 <模型目录名>_mean，后续脚本以 --model bert-base-chinese_mean 复用同一条代码路径，两套结果互不覆盖。运行日志由 shell 脚本重定向到 results/*.log，脚本自身不写日志；run_md1_all.sh、run_extras.sh、run_extras_resume.sh、run_distance_means.sh 是实际批量跑出本仓库结果用的驱动脚本（后三个还把 results/*.json 备份到 D 盘）；run_distance_means.sh 只做一件事——把三个单语模型均值池化的距离控制串行跑完，随后自动调用 summarize.py 与 plot_results.py。

3.5.2 数据层

CoNLL-U 解析 (treebank.py)。用 conllu.parse_incr 流式读取；words[i] = token["form"]（词形，不是 lemma，中文树库中两者常不同，如"他们"的 lemma 是"他"）。父节点取 heads = token["head"] - 1，把 UD 的 1-based 列转为 0-based，于是根的 head = 0 变成 -1。距离矩阵由 _distances_and_depths 计算：先按 $h \geq 0$ 建无向邻接表（每条边加双向），再对每个起点跑一次 BFS (collections.deque)，得到 **(n,n)** 的无向最短路矩阵，即两词在依存树上的路径长度；深度则以 heads < 0 定位唯一根后再跑一次 BFS，得到每词到根的跳数（根为 0）。**标点不剔除**：函数名与 docstring 明确写为 *_incl_punct，标点词在 UD 里是普通 token（有 head、deprel = punct），因此进入索引、距离矩阵、深度向量与后续全部评估（实测 train 中 punctuation 词 13,627 / 98,614）。**根**同样按普通节点处理，只是没有父边、深度恒为 0，故不进入无向边集合。多词 token（形如 1-2）与空节点（形如 3.1）在三个文件里各出现 0 次，代码也没有为它们写分支——解析完全依赖 conllu 的默认迭代（代码中未体现对多词 token/空节点的显式处理）。解析结果还带一句断言：距离矩阵所有元素 ≥ 0 ，即"树应当连通"。每句另存 text = sent.metadata["text"]，供字符级对齐使用（实测 4,997 句都有该字段）。

基线 (baselines.py)。线性链把边集设为 $(i,i+1):0 \leq i < n-1$ ，预测距离直接取 $|i-j|$ （末词自然成为链的端点，代码未显式声明根）。随机树对每句生成 N_RANDOM = 10 棵均匀随机标号树：_random_tree_edges 用 Prüfer 序列 (rng.integers(0, n, size=n-2) 加度数表逐步取"索引最小的叶子"解码)，再由 scipy.sparse.csgraph.shortest_path(..., directed=False) 得到距离矩阵。句内 10 棵树先各自算 UUAS/Spearman 再取平均，然后按句平均；评估口径与 probe.py 完全一致（n<2 跳过、无向边、上三角 Spearman、剔除 NaN）。种子固定为 0，且每个 split 各自重新初始化 np.random.default_rng(0)。

3.5.3 表示层：抽取与中文字↔词对齐

extract.py 的流程是"整批分词 → 整批前向 → 逐句对齐 → 逐词池化 → 写 npz"。

- 分词与批前向**：AutoTokenizer.from_pretrained(<本地目录>)，一批 16 句 (--batch-size, 默认 16) 一次调用 tok(texts, padding=True, return_tensors="pt", return_offsets_mapping=True)；**未传 truncation**（代码中未启用截断；本树库最长句 111 词 / 182 字符，远低于 512，故未触发长度问题，但代码本身没有

截断或分块的兜底)。模型用 `AutoModel.from_pretrained(...).eval()` (不是 `ForMaskedLM`, 因此没有 `lm_head` 的 logits), `extract_split` 整个函数被 `@torch.no_grad()` 装饰; 权重从不更新——代码没有显式写 `requires_grad_(False)`, 冻结由"只前向、不建优化器"实现。

- **13 个抽取点**: `output_hidden_states=True` 返回 13 个张量, `n_layers = model.config.num_hidden_layers + 1`。键名 `layer_0 ... layer_12`, 其中 `layer_0` 是嵌入层 (词嵌入 + 位置/段嵌入 + `LayerNorm`) 的输出, `layer_1...12` 是 12 层 Transformer 的逐层输出。
- **对齐**: `align_words` 在 # text 上按词形顺序匹配, 先试 `text.startswith(w, cursor)`, 失败则跳过空白再试, 再失败就用 `text.find(w, cursor)` 全文回查并计入"失配再找回"; 仍未找到则抛 `ValueError` (不是静默跳过)。子词映射由 `word_token_indices` 完成: 取所有与词形字符区间 $[w_s, w_e]$ 有重叠的非特殊子词; 特殊符号由 `tokenizer` 给出 (0,0) offset, 据此把 `[CLS]/[SEP]/[PAD]` 排除在池化之外。
- **两种池化**: `--pooling first` (默认) 取每个词的 `ix[0]`, 即第一个重叠子词——对逐字分词的 BERT 家族就是"词首字"; `--pooling mean` 取 `h[b, ix].mean(0)`, 即词内全部子词的均值。
- **存储**: `layers[li][row:row+n] = word_h.half().numpy()`, 即 float16 落盘, 最后 `np.savez(dest / f"{split}.npz", **out)` (savez 不压缩)。npz 键为 `layer_0...layer_12` (形状 (总词数, 768), float16) 与 `lengths` (形状 (n_sents,), int32)。lengths 的语义是"把行方向按句切分", `lengths.sum()` 必须等于树库总词数; 切句由 `probe.split_sentences` 用 `np.cumsum` 完成。
- **两个 fallback 的触发条件**: ①"失配再找回"——词形在当前位置 (含跳空白) 匹配不上, 但在 # text 更后面能定位 (如词间空白与 `SpaceAfter` 缺失不一致); ②"无子词覆盖回退"——某词的字符区间与任何非特殊子词都无重叠 (词形被 `tokenizer` 的 `normalizer` 剥离, 如组合音符), 此时退取该词之后的第一个非特殊子词, 若其后无子词则退到整句最后一个非特殊子词。两个计数只打印 (失配再找回: N 词, 无子词覆盖回退: M 词), 不写入 npz、不做断言。现存日志中失配再找回全为 0 (0 例); 零覆盖回退在 MacBERT (首字与均值) 与 RoBERTa-wwm-ext (均值) 的 train 上各为 1 词, 其余为 0 (RoBERTa 首字池化那一次的抽取日志不在仓库里, 代码中未体现)。

3.5.4 探针层

网络结构。DistanceProbe 与 DepthProbe 都只有一个参数 $B \in \mathbb{R}^{\{\text{rank} \times 768\}}$, 用 `torch.randn(rank, dim) * 0.01` 初始化, 不带 bias。前向先用 `torch.einsum("rd,bnd->bnr", B, h)` 把词投影到 rank 维, 距离探针再取 $(u_i - u_j)^2$ 在 rank 维求和——与 $\|B(h_i - h_j)\|^2$ 严格等价 (线性性), 但避免了在 768 维上构造 n^2 个差向量; 深度探针取 $\|u_i\|^2$ 。秩 128 只体现在这一处: B 的行数就是 128, 因此 $\|B(h_i - h_j)\|^2 = (h_i - h_j)^T M (h_i - h_j)$ 中的 $M = B^T B$ 秩至多 128。

损失。严格 L1, 按批内有效元素取均值: 距离探针 `loss = (pred - gold).abs()[pair_mask].mean()`, 其中 `pair_mask = mask ⊗ mask` (mask 由批内真实句长构造, padding 位置被排除); 这覆盖批内全部有效有序词对, 含对角元素 (i, i) ——该项 gold 与预测都为 0, 对均值只起稀释作用。深度探针用 `(pred - gold).abs()[mask].mean()`。因此损失是"按批内有效词对 (有效词) 取均值", 不是按句分摊: 含长句的批与含短句的批权重由批内元素数决定。

批处理与优化。make_batches 每 epoch 用 `np.random.default_rng(seed).permutation` 重排句序, 再贪心装箱: 每批句数 $\leq \text{batch_size} = 32$, 且批内 $n_{\text{max}}^2 \times \text{句数} \leq \text{pair_budget} = 120000$ (用平方量级预估内存), 因此长句自动落进小批。优化器是 `torch.optim.Adam(lr = 1e-3)` 加 `CosineAnnealingLR(T_max = epochs = 20)`; train_probe 内部先 `torch.manual_seed(seed)` 再新建 `default_rng(seed)`。表示本身是常数 (`torch.from_numpy` 得到的 float32 张量), 梯度只流过 B, 所以不存在"表示适应探针"的路径。

评估。eval_distance_probe 逐句计算, `n<2` 跳过: 把预测距离矩阵送入 `scipy.sparse.csgraph.minimum_spanning_tree`, 取 `tocoo()` 的 (row, col) 作为预测无向边集合; gold 无向边

集合由 $\text{heads} \geq 0$ 构造。关于标点与根：**代码不剔除标点**——标点词参与 MST（可成为内部节点），其依存边也计入 gold 集合；只有根（ $\text{head} = -1$ ）不产生 gold 边，因此分母恒为句长减一；若根被预测为某个词的子节点，这条边不在 gold 集合里，会直接缩小交集。 $\text{UUAS} = |\text{预测边} \cap \text{gold 边}| / |\text{gold 边}|$ ，按句平均。Spearman 取每句上三角（ $i < j$ ）的预测距离与 gold 距离，`scipy.stats.spearmanr(...).statistic`，按句平均并剔除 NaN。`eval_depth_probe` 同理：逐句 `spearmanr(预测深度, gold 深度)`（全体词，含标点）后按句平均；根准确率为 $\arg \min(\text{pred}) = \arg \min(\text{gold})$ （预测深度最小者应为根）的比例。主线脚本对每层训练两个独立探针，且构造每个探针前都重设 `torch.manual_seed(args.seed)`，两者共用同一个 seed。

3.5.5 控制任务

随机假深度。 `depth_control_labels` 先取 `splits` 的**第一个键**（`run_control.py` 按 train/dev/test 顺序构造字典，故即 train）中所有句子的最大深度 `max_depth`，再维护一张“词形 $\rightarrow$ 整数”的表：首次遇到某个词形时用 `np.random.default_rng(seed).integers(0, max_depth + 1)` 均匀抽取并永久固定。因为按 split 顺序逐句逐词遍历，随机流的消耗顺序完全由 splits 顺序决定，结果确定；又因为表是跨 split 共享的，dev/test 中 train 未见过的词型也会拿到一个固定标签。注意这个目标的可解性来自“词型身份”而非句法，这正是控制任务的意义。

随机树。 `distance_control_trees` 对每句用 `_random_heads(n, rng)` 生成一棵 Prüfer 序列的均匀随机标号树： $n = 1$ 、 $n = 2$ 特判，解码时每步取索引最小的叶子，最后连接剩下的两个度 1 节点；随后从词 0 出发 BFS 定向得到 0-based 父节点（**根固定为词 0**，与真树的根位置无关）。距离矩阵用 `scipy.sparse.csgraph.shortest_path(..., directed=False)` 计算。节点数、句长、词序与真树完全一致，唯一差别是拓扑；train/dev/test 的随机树互相独立（同一条 rng 流按 split、句序依次抽取），由 `seed = 0` 保证确定性。

与主实验的共用部分。 两条控制线都直接复用 `probe.py` 的 `DistanceProbe/DepthProbe`、`train_probe`、`eval_distance_probe/eval_depth_probe`，以及 `run_first_probe.load_split` 的数据装载与断言，超参也完全一致（rank 128 / epochs 20 / lr $1e-3$ / batch_size 32 / seed 0）；被替换的只有“训练目标”（真树/真深度 $\rightarrow$ 随机树/假深度）。`run_control.py` 在每层结束时若发现 `results/first_probe_<model>.json` 存在，就打印该层的 `selectivity = gold 指标 - 控制指标` 作为人工核对；汇总表里的 `selectivity` 由 `summarize.py` 从两份 JSON 现算。

3.5.6 MDL 检验

标签。 `run_md1.deprel_labels` 从 train 的排序去重标签集建词表（45 类，含 case:loc 之类的汉语子类），逐句展平为索引；test 使用同一词表，未登录标签映射为 -1，而训练出的分类器永远不会输出 -1，故这些词在参考准确率里必错（实测四个模型的 test 未见标签数均为 0，该分支只是保险，未真正触发）。

前缀切分。 `PORTIONS = [0.1, 0.2, ..., 1.0]`；`order = np.random.default_rng(seed).permutation(len(X))` 把 train 的全部词按这个**固定随机序**重排，再切出 `bounds = [0] + [round(t  $\times$  N)]`（ $N = 98,614$ ）。同一 seed 使各层使用同一个排列，层间可比。

码长累加。 首块 `[0, bounds[1]]` 没有任何可用信息，用均匀码，代价 $(\text{bounds}[1] - \text{bounds}[0]) \times \log_2(K)$ ；实测 $K = 45$ 时该块为 54,156 bit，全量均匀码长 $\text{uniform_bits} = N \times \log_2 K = 541,574$ bit。其后对 $k = 1 \dots 9$ ：用此前**全部**数据 `Xp[:bounds[k]]` 训练一个 `torch.nn.Linear(768, 45)`（默认带 bias），损失为交叉熵、优化器 Adam(lr = $1e-3$)、每前缀固定 5 个 epoch、batch_size = 1024（无早停、无验证集、不用 dev）；再用它的 `log_softmax` 对本块金标签累加 $-\log_2 p(\text{gold})$ ，实现是 `nll = -logp(range(len(block)), y_block).sum()`；`bits += float(nll) / float(np.log(2))`（`log_softmax` 为自然对数，除以 **ln 2** 换算成 bit）。最后一块（0.9 $\rightarrow$ 1.0）的编码概率来自只用前 90% 数据训练的探针。

报告口径。 总码长 `bits` 只在 train 上累计；压缩率 `compression = uniform_bits / bits`；同时报告最后一个探针在 test 上的准确率 `test_acc`（accuracy 按 8192 一批算 `argmax` 命中率）作参考。`train_classifier` 每次

调用先 `torch.manual_seed(seed)` 再 `default_rng(seed)`，所以各前缀的探针初始化与批内打乱顺序完全相同，只有数据不同。

uniform_selftest。直接运行 `python src/md1.py` 会执行该自检：在 $n = 2000$ 、 $\dim = 16$ 、 $k = 7$ 的合成数据上，用“零权重线性层”（输出严格均匀分布）替换训练函数，验证在线码长与 $n \log_2 k$ 的相对误差 $< 1e-3$ ，打印 UNIFORM GATE: PASS/FAIL 并以退出码区分。它检验的是**码长累加公式与 bit 换算**在与均匀码应当等价的情形下确实等价，不涉及真实表示。

3.5.7 工程稳健性与可复现性

- **逐层 checkpoint + 原子替换**：三个实验脚本（`run_first_probe.py`、`run_control.py`、`run_md1.py`）都定义了 `checkpoint()`，每完成一层就把“元信息 + 全部已完成行”写成 `<dest>.tmp`，再用 `tmp.replace(dest)` 覆盖正式文件——同目录替换，中断不会留下半个 JSON。
- **断点续跑**：启动时若目标 JSON 已存在，读出 `layers` 得到已完成层号集合，主循环直接 `if layer in done: continue`；`run_control.py` / `run_md1.py` 另有 `--layers` 参数可只跑指定层做快速自检。
- **随机种子**：`torch.manual_seed(seed)`（探针/分类器初始化与 `torch` 侧批序）+ `np.random.default_rng(seed)`（句序打乱、词型随机深度、Prüfer 序列），默认 `seed = 0`，命令行可覆盖。
- **CPU 多线程确定性**：`src/` 下没有任何 `torch.set_num_threads`、`OMP_NUM_THREADS` 或 `torch.use_deterministic_algorithms` 设置，也没有固定 `PYTHONHASHSEED`——即**未做特殊设置**，逐位可复现性依赖 CPU 算子本身的确定性。
- **fp16 与体积**：`npz` 以 `float16` 落盘（训练时统一 `.astype(np.float32)`）。单文件体积 = 词数 $\times$ 13 层 $\times$ 768 $\times$ 2 字节：`results/representations/bert-base-chinese/` 下 `train.npz` ≈ 1.97 GB（98,614 词）、`dev.npz` ≈ 241 MB、`test.npz` ≈ 229 MB；7 个表示目录合计约 16.6 GB（代码未注释说明选 `fp16` 的理由；直接效果是把磁盘占用减半）。
- **耗时量级**（读各 JSON 的 `seconds` 字段，`bert-base-chinese`）：主线 13 层合计 5,261.5 s ≈ 88 min（单层 402–408 s）；深度控制 716.5 s ≈ 12 min（单层 53–67 s）；距离控制 4,818.4 s ≈ 80 min（单层 369–374 s）；MDL 172.0 s ≈ 3 min（单层 13–15 s）；mBERT 主线 5,414.6 s，与单语模型同量级。

3.5.8 复现步骤与环境

以下命令在仓库根目录的 Git Bash 下按序执行（Windows 路径风格，`PYTHONIOENCODING=utf-8` 用于中文输出）：


```

# 0) 环境
python -m venv .venv
./venv/Scripts/python -m pip install -r requirements.txt
./venv/Scripts/python -m pip install torch --index-url https://download.pytorch.org/whl/cpu

# 1) 树库 (jsDelivr 通道, 见 docs/feasibility.md; 仓库内没有下载脚本, 路径按数据目录实际文件名)
mkdir -p data/ud_gsdsimp
for s in train dev test; do
    curl -L -o "data/ud_gsdsimp/zh_gsdsimp-ud-$$s.conllu" \
        "https://cdn.jsdelivr.net/gh/UniversalDependencies/UD_Chinese-GSDSimp@master/zh_gsdsimp-ud-$$s.conllu"
done
./venv/Scripts/python src/treebank.py data/ud_gsdsimp/zh_gsdsimp-ud-train.conllu # 应打印 3997 句

# 2) 模型 (ModelScope 直链 -> models/<名称>/)
./venv/Scripts/python src/download_model.py AI-ModelScope/bert-base-chinese
./venv/Scripts/python src/download_model.py dienstag/chinese-roberta-wwm-ext
./venv/Scripts/python src/download_model.py dienstag/chinese-macbert-base
./venv/Scripts/python src/download_model.py AI-ModelScope/bert-base-multilingual-cased

# 3) 表示抽取: 四个模型 (首字池化) + 三个单语模型 (均值池化)
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    ./venv/Scripts/python src/extract.py "models/$m" --pooling first
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    ./venv/Scripts/python src/extract.py "models/$m" --pooling mean
done

# 4) 过拟合门 + 基线
./venv/Scripts/python src/run_first_probe.py --overfit64 --model bert-base-chinese --layer 7
./venv/Scripts/python src/baselines.py

# 5) 主线探针 (四模型首字 + 三模型均值)
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    ./venv/Scripts/python src/run_first_probe.py --model "$m"
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    ./venv/Scripts/python src/run_first_probe.py --model "${m}_mean"
done

# 6) 控制实验 (深度 + 距离; 四个模型首字 + 三个单语模型均值)
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    for c in depth distance; do
        ./venv/Scripts/python src/run_control.py --control "$c" --model "$m"
    done
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    for c in depth distance; do
        ./venv/Scripts/python src/run_control.py --control "$c" --model "${m}_mean"
    done
done

# 7) MDL (含自检)
./venv/Scripts/python src/mdl.py
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    ./venv/Scripts/python src/run_md1.py --model "$m"
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    ./venv/Scripts/python src/run_md1.py --model "${m}_mean"
done

# 8) 汇总与绘图 (只读 results/*.json)
./venv/Scripts/python src/summarize.py # -> docs/results-tables.md
./venv/Scripts/python src/plot_results.py # -> figures/fig1..4.png

```

本仓库实际使用的批量驱动脚本有四个：bash run_md1_all.sh（三模型 MDL 串行）、bash run_extras.sh（mBERT 对照 + 均值池化消融，共 9 个阶段；中断后用 bash run_extras_resume.sh 从阶段 7 续跑）与 bash run_distance_means.sh（三个单语模型均值池化的距离控制，逐层断点续跑，跑完自动汇总与绘图）。这四个脚本都把输出追加写到 results/*.log。

环境（来自 ./venv/Scripts/python -m pip freeze）：Python 3.13.9; torch 2.14.1+cpu; transformers 5.18.0; tokenizers 0.23.2; conllu 6.0.0; numpy 2.5.3; scipy 1.18.1; matplotlib 3.11.2; safetensors 0.8.0; huggingface_hub 1.33.0。硬件为单台笔记本 CPU（无 GPU，未使用 CUDA）。

3.5.9 验收门（跑通性自检）

检查项	位置	判据与形式
解析句数 3,997 / 500 / 500	src/extract.py 主流程逐 split 打印 [train] 3997 句, 98614 词; src/treebank.py 直跑也打印	打印项，不是断言
npz 词数与树库一致	src/extract.py 的 assert out["lengths"].sum() == n_words; src/run_first_probe.load_split 再断言一次（三个实验脚本共用）	断言，失败即中止
对齐失配 / 零覆盖计数	src/extract.py 打印两计数	打印项，不落盘、不断言
批内词数守恒	extract_split 的 assert row == total_words	断言
树连通性	treebank._distances_and_depths 断言 BFS 距离全 ≥ 0; control_tasks._random_heads 断言随机树连通	断言
--overfit64 门	src/run_first_probe.py::run_overfit64	在 train 前 64 句、同一层上训练并评估距离探针 (batch_size = 8, 层默认 7) ; Spearman > 0.9 才打印 OVERFIT GATE: PASS 并以退出码 0 结束，否则 FAIL 且退出码 1。它验证"优化流程确实能拟合给定信号"，与表示质量无关
MDL uniform 自检	src/mdl.py::uniform_selftest (python src/mdl.py)	均匀探针下在线码长与均匀码长相对误差 < 1e-3 → UNIFORM GATE: PASS, 否则 FAIL 退出码 1
UUAS 落在合理区间	无自动断言	参照物是 results/baselines.json (dev: 随机树 0.0960、线性链 0.4256)，属人工检查（代码中未体现自动检查）

另外两处"打印式自检"：train_probe 每 5 个 epoch 打印 batch 平均 L1；run_control.py 每层打印 [检查] ... 选择性应为正，供人眼核对。

3.5.10 实现取舍与局限

- **首字池化是默认**（--pooling 默认 first），均值池化只作消融；两种池化共用同一函数，差别仅在一个分支（ix[0] 对 h[b, ix].mean(0)）。
- 距离探针**不含 bias、只有一个 rank = 128 的矩阵**，秩 128 硬编码为默认值，代码里没有秩的扫描或选择流程；MDL 的分类器则用默认 nn.Linear（含 bias）。
- **距离控制不按词型条件化**（control_tasks.py docstring 明确说明这是设计选择）：整棵树不是词级标签，逐词型条件化没有标准做法。
- **随机假深度取 [0, max_depth] 上的均匀整数**，与真实深度的分布（集中在浅层）并不一致；max_depth 只由 train 决定。
- MDL 的**码长只在 train 上累计**，dev 完全未参与；每个前缀固定 5 epoch、无早停、无超参搜索；最后一块的编码概率来自只用前 90% 数据训练的探针；压缩率的分母 uniform_bits 是全量 train 的均匀码长。
- **test 中未见 deprel 映射为 -1** 只会压低参考准确率、不影响码长；本数据上该情况为 0 例。
- **单一 seed**（默认 0）、**单一树库**（代码只读 data/ud_gsdsimp/）；data/ud_pud/zh_pud-ud-test.conllu 存在于数据目录，但没有任何脚本引用它（代码中未接入）。
- **分词未启用 truncation**：代码没有任何截断或分块兜底，超过模型上限的句子只能让前向失败（本树库最长句 111 词，实际未触发）。

- 均值池化消融只覆盖**三个单语模型**，mBERT 没有均值池化的一组（results/first_probe_bert-base-multilingual-cased_mean.json 不存在）。summarize.py 与 plot_results.py 只在文件存在且层数齐全时才输出对应的行与曲线，缺数据的模型会被跳过（plot_results.py 对图 4(c) 会打印一行跳过提示）。
- 表与图一律由 results/*.json **现算**（每层值、峰值层、最小值、耗时都是汇总时算出的），代码中没有手工填写的中间数字。

4 结果

4.1 RQ1：句法几何的存在与位置

三个模型、13 个抽取点的 dev 指标全表见表 1，test 见表 2，曲线见图 1。

表 1 dev (500 句) 句法几何指标

层	BERT UUAS	BERT dSpr	BERT hSpr	BERT 根	RoBERTa UUAS	RoBERTa dSpr	RoBERTa hSpr	RoBERTa 根	MacBERT UUAS	MacBERT dSpr	MacBERT hSpr	MacBERT 根
0	0.375	0.525	0.465	0.168	0.375	0.527	0.460	0.140	0.378	0.527	0.461	0.144
1	0.394	0.585	0.626	0.242	0.399	0.590	0.648	0.236	0.401	0.591	0.650	0.238
2	0.444	0.641	0.702	0.304	0.450	0.654	0.716	0.274	0.455	0.656	0.720	0.300
3	0.522	0.706	0.745	0.360	0.534	0.717	0.761	0.330	0.540	0.718	0.764	0.308
4	0.538	0.725	0.769	0.396	0.558	0.735	0.780	0.362	0.558	0.737	0.786	0.382
5	0.550	0.729	0.773	0.396	0.567	0.738	0.785	0.380	0.558	0.730	0.793	0.378
6	0.552	0.732	0.781	0.410	0.578	0.745	0.792	0.384	0.566	0.742	0.796	0.350
7	0.579	0.752	0.796	0.402	0.583	0.756	0.800	0.402	0.585	0.760	0.804	0.362
8	0.587	0.751	0.799	0.426	0.587	0.757	0.803	0.420	0.589	0.762	0.803	0.370
9	0.559	0.732	0.790	0.354	0.564	0.735	0.792	0.394	0.562	0.741	0.790	0.342
10	0.535	0.709	0.779	0.368	0.539	0.711	0.784	0.384	0.539	0.717	0.784	0.358
11	0.513	0.690	0.768	0.318	0.517	0.689	0.777	0.350	0.514	0.690	0.777	0.340
12	0.487	0.676	0.762	0.368	0.485	0.672	0.774	0.308	0.475	0.666	0.756	0.224

(dSpr = 距离 Spearman, hSpr = 深度 Spearman, 根 = 根节点准确率。基线：随机树 dev UUAS 0.096 / Spearman -0.001；线性链 0.426 / 0.388。)

表 2 test (500 句) UUAS / 距离 Spearman

层	BERT	RoBERTa	MacBERT
0	0.385 / 0.533	0.393 / 0.536	0.390 / 0.537
3	0.532 / 0.722	0.543 / 0.733	0.544 / 0.736
6	0.568 / 0.745	0.588 / 0.759	0.588 / 0.758
7	0.587 / 0.759	0.598 / 0.770	0.595 / 0.774
8	0.596 / 0.763	0.605 / 0.769	0.597 / 0.771
9	0.563 / 0.743	0.572 / 0.747	0.577 / 0.750
10	0.546 / 0.720	0.545 / 0.726	0.553 / 0.728
12	0.499 / 0.683	0.495 / 0.680	0.485 / 0.676

(完整 13 层 × 3 模型表见 docs/results-tables.md。)

发现 1：汉语句法在表示中是可解码的，且强度可观。 峰值层 dev UUAS 0.587–0.589、test 0.596–0.605，距离 Spearman 0.75–0.77，深度 Spearman 约 0.80、根准确率 0.37–0.43。相比之下，把随机树当目标时的 UUAS 只有 0.096（与随机猜测持平），线性链基线 0.426——低于任何一个模型的第 2 层以上。也就是说，模型表示中确实存在一个能被线性读出恢复的树结构，且它远好于任何"与语言学无关的基线树"。

发现 2：峰值同在第 8 层（共 13 个抽取点，即倒数第 5 层），呈驼峰形。 UUAS 在三个模型上均于第 8 层达到最大；距离 Spearman 的峰值在层 7–8（BERT 层 7 的 0.752 与层 8 的 0.751 基本持平，MacBERT/RoBERTa 峰值在层 8）；深度 Spearman 在层 7–8 达 0.80。此后 UUAS、距离 Spearman 与深度

Spearman 均随层单调下降：到第 12 层，UUAS 掉到 0.475–0.487，深度 Spearman 掉到 0.756–0.774（根准确率是 500 句上的二值指标、噪声大，上下波动，不受这一趋势约束）。

发现 3：输入层（层 0）就已经有相当高的"表面"可解码性。 层 0 的 UUAS 已达 0.375–0.378、距离 Spearman 0.525–0.527，这来自字向量本身与位置编码（词序信息）——它恰好说明为什么必须做控制检验：高指标未必是句法。

4.2 RQ2：预训练策略与模型来源的影响

(a) 三种遮蔽策略殊途同归。 图 1 (figures/fig1_syntax_geometry.png) 把三条单语模型曲线画在一起，结论相当干脆：

- **形状一致：**三条曲线在所有层上的起伏形态几乎重合，都是"层 0–3 快速上升 → 层 4–7 缓慢爬升 → 层 8 达峰 → 之后下降"。
- **峰值同层：**三模型 UUAS 峰值都在第 8 层，dev 值 0.587 / 0.587 / 0.589（相差 0.2 个百分点），test 0.596 / 0.605 / 0.597。
- **强度差很小且不一致：**全层最大差距 2.6 个百分点（层 6，RoBERTa 0.578 对 BERT 0.552）；多数层差距在 1 个百分点上下，且排序并不稳定：层 0–4 与层 7–8 由 MacBERT 略高（差 0.4–2.0 个百分点），层 5–6 反过来由 RoBERTa 略高（差 1.6 与 2.6 个百分点），末层 BERT 略高。MDL 里排序还会翻转：第 8 层最低码长是 BERT 157.1 kbit，RoBERTa 157.9 kbit。
- **深度（层级结构）与距离（依存结构）两类目标给出同一图景；**MacBERT 的深度 selectivity 略高（见 4.3），是唯一一处稳定可察的模型间差异。

因此 RQ2 的第一个答案是：**预训练遮蔽策略（原版 MLM / wwm / 纠错式）改变的是下游可用性，而不是句法几何本身——**三种训练目标殊途同归到同一层、同一强度、同一形状的句法几何。需要诚实指出：我们只跑了单个 seed，1 个百分点量级的排序差异无法与种子噪声区分开；可以放心主张的是"形状与峰值层一致"这一稳健结论，而不是"MacBERT 比 BERT 强 1 个点"。

(b) 多语 mBERT 不比单语模型弱，但峰值早一层。 我们在完全相同的流水线、树库与超参下补跑了 mBERT（表 6、表 7）。

表 6 mBERT (bert-base-multilingual-cased) 逐层指标

层	dev UUAS	dev dSpr	dev hSpr	test UUAS	test dSpr	深度 Δ (dev)	距离 Δ (dev)	距离控制 UUAS
0	0.367	0.520	0.473	0.379	0.526	−0.150	0.271	0.095
1	0.373	0.546	0.583	0.385	0.563	−0.111	0.280	0.093
2	0.407	0.587	0.643	0.417	0.600	−0.115	0.310	0.097
3	0.484	0.679	0.731	0.499	0.694	−0.047	0.387	0.097
4	0.526	0.711	0.753	0.546	0.726	−0.022	0.432	0.094
5	0.549	0.723	0.774	0.568	0.740	0.000	0.457	0.093
6	0.592	0.759	0.796	0.611	0.771	0.044	0.494	0.097
7	0.604	0.770	0.806	0.619	0.778	0.072	0.502	0.102
8	0.592	0.764	0.808	0.599	0.768	0.095	0.492	0.100
9	0.575	0.751	0.800	0.589	0.756	0.102	0.476	0.099
10	0.560	0.730	0.794	0.576	0.736	0.098	0.466	0.095
11	0.535	0.705	0.782	0.551	0.712	0.096	0.442	0.092
12	0.475	0.646	0.759	0.487	0.656	0.104	0.376	0.099

表 7 四模型峰值对比

模型	UUAS 峰值层	dev UUAS	dev dSpr	test UUAS	test dSpr	dev hSpr 峰值	层 8 深度 Δ	MDL 最低层	MDL 最低 kbit
BERT	8	0.587	0.751	0.596	0.763	0.799 (层 8)	+0.071	8	157.1
RoBERTa-wwm-ext	8	0.587	0.757	0.605	0.769	0.803 (层 8)	+0.072	8	157.9
MacBERT	8	0.589	0.762	0.597	0.771	0.804 (层 7)	+0.113	8	160.7
mBERT (多语)	7	0.604	0.770	0.619	0.778	0.808 (层 8)	+0.095	7	158.3

mBERT 在四个指标上都略高于三个单语模型，而且 **UUAS 峰值提前到第 7 层**：距离 selectivity（层 7 的 0.502 对层 8 的 0.492）与 MDL 最低码长（层 7 的 158.3 对层 8 的 159.6 kbit）也都落在层 7，三条线方向一致；唯一没有跟着提前的是深度 Spearman 峰值（仍在层 8，0.808）。需要诚实说明：这三处差距都在 1 个百分点上下，单看任何一条都落在种子噪声量级（见第 5 节局限①），值得信任的是三条线方向一致这一点，而不是任何单条的差值。这个结果与常见印象（“多语模型的中文更弱”）方向相反，但两条主张并不冲突：Chi et al. (2020) 发现的是中文在 **mBERT 内部** 相对其他语言编码偏弱（一个跨语言相对比较），我们测的是**跨模型的绝对可解码性**（中文单语模型 vs mBERT 在中文上的表现）。一个模型可以在自己的语言谱系里相对偏弱，同时仍然优于专门训练的中文单语模型——我们的数据显示后者成立。

控制检验排除了“mBERT 只是词形记得更多”这一解释：它的第 8 层深度 selectivity 为 +0.095，落在三个单语模型的区间（+0.071/+0.072/+0.113）之内；随机树控制的 UUAS 全层在 0.092–0.102，与随机水平无异；低层同样出现负 selectivity（层 0 为 -0.150），转正位置也在层 5 附近。也就是说，mBERT 的中层句法几何与单语模型是同一类东西，只是更早显现。

(c) 消融：结论不依赖中文特有的字↔词对齐方式。 把词表示从“词首子词”改为“词内均值”后（表 8）：

表 8 均值池化消融（与首字池化对比）

模型	首字池化峰值	均值池化峰值	层 8 深度 Δ（首字 → 均值）	层 8 距离 ΔUUAS（首字 → 均值）	MDL 最低 kbit（首字 → 均值）
BERT	层 8, 0.587	层 8, 0.617	+0.071 → +0.087	0.491 → 0.519	157.1 → 150.6
RoBERTa-wwm-ext	层 8, 0.587	层 8, 0.611	+0.072 → +0.083	0.489 → 0.512	157.9 → 151.3
MacBERT	层 8, 0.589	层 8, 0.613	+0.113 → +0.131	0.492 → 0.519	160.7 → 154.3

四件事同时成立：**UUAS 峰值层完全不变（三模型都仍是 8）**；绝对值整体上移约 3 个百分点（层 8 的 dev UUAS 0.587 / 0.587 / 0.589 → 0.617 / 0.611 / 0.613）；**深度与距离两类 selectivity 都同步上移**（层 8 深度 Δ +0.071 / +0.072 / +0.113 → +0.087 / +0.083 / +0.131；层 8 距离 ΔUUAS 0.491 / 0.489 / 0.492 → 0.519 / 0.512 / 0.519）；**MDL 最低码长同时缩短**（157.1 / 157.9 / 160.7 → 150.6 / 151.3 / 154.3 kbit）而最低层不变。关键在于控制任务一侧没有跟着上移——随机树控制的 dev UUAS 全层仍是 0.091–0.104、与基线 0.096 无异，说明上移来自真结构信号被读得更干净，而不是更强的词形记忆。距离 selectivity 的峰值层在均值池化下仍是层 7–8（BERT 与 MacBERT 在层 8，RoBERTa 层 7 的 0.517 略高于层 8 的 0.512，差 0.5 个百分点，属噪声量级）。均值池化只是给了探针更干净、噪声更小的词表示，句法几何的位置与形状不受影响。因此本文的核心结论对中文特有的对齐处理是稳健的。

4.3 RQ3：控制检验下的存活

(a) 深度控制：低层的可解码性主要是词形记忆。 表 3 与图 2(a) 给出逐层 selectivity。

表 3 深度探针 selectivity（dev, gold 深度 Spearman – 控制深度 Spearman）

层	BERT Δ	BERT 控制	RoBERTa Δ	RoBERTa 控制	MacBERT Δ	MacBERT 控制
0	-0.153	0.619	-0.161	0.621	-0.158	0.619
1	-0.147	0.773	-0.146	0.794	-0.145	0.795
2	-0.097	0.799	-0.091	0.807	-0.085	0.805
3	-0.056	0.801	-0.043	0.803	-0.039	0.802
4	-0.019	0.788	-0.004	0.784	+0.006	0.780
5	+0.001	0.772	+0.021	0.764	+0.038	0.755
6	+0.028	0.753	+0.046	0.746	+0.065	0.731
7	+0.060	0.736	+0.063	0.737	+0.094	0.710
8	+0.071	0.729	+0.072	0.730	+0.113	0.690
9	+0.068	0.722	+0.061	0.731	+0.100	0.690
10	+0.071	0.708	+0.044	0.741	+0.095	0.689
11	+0.065	0.703	+0.041	0.736	+0.099	0.678
12	+0.054	0.709	+0.032	0.741	+0.074	0.682

三个模型呈现同一种模式。在最底层，控制任务的分数**高于**真任务（层 0 控制深度 Spearman 0.62 对 gold 0.46，selectivity -0.15 上下）：说明此时表示里最容易读出的东西是"这个词是什么"（词型身份），而随机假深度恰好把"同一词型的每一次出现赋同一个值"，是词型身份可以完美解决的问题；真实树深度反而要求计算结构。控制分数在层 1–3 达到 0.77–0.81 的峰值后下降。selectivity 在层 4–5 由负转正，随后上升到**与句法峰重合于第 8 层**（BERT +0.071、RoBERTa +0.072、MacBERT +0.113），再随层回落。

这条结果的解释力在于：如果第 8 层的"句法峰"只是词形信息的副产品，selectivity 应当在第 8 层最负或至少不峰；实际却是第 8 层 selectivity 与探针指标同时达峰。因此**"中层存在句法几何"这一主张在控制检验下存活**，而"低层已能解码句法"的说法被证伪——低层的高指标必须打折。MacBERT 的深度 selectivity 明显高于另两个模型（第 8 层 0.113 对 0.071/0.072），这是三模型间唯一一稳定可察的差别，可能与它的纠错式目标（用近义词替换被遮蔽的词、促使模型更依赖上下文而非词表身份）有关，但单 seed 下不宜过度解读。

(b) 距离控制：随机结构进不了表示。表 4 与图 2(b)：以固定随机树为目标的探针，dev UUAS 全层落在 0.093–0.104，与随机树基线 0.096 无法区分；dev Spearman 三模型全层 $|\rho| \leq 0.01$ 。相反，真树探针的 selectivity 从层 0 的 0.279 单调升到第 8 层的 0.491（三模型 0.489–0.492），随后回落到 0.386–0.387。这一结论不随池化方式改变：换用词内均值后（图 4(c)），随机树控制的 dev UUAS 仍落在 0.091–0.104（基线 0.096）、dev Spearman 全层 $|\rho| \leq 0.011$ ，而层 8 的 selectivity 升到 0.512–0.519（首字 0.489–0.492）。控制分数没动、真树分数上移，正是"上移来自更干净的真结构信号"的直接证据。

值得注意的是一个"负面对照"本身很有说服力：随机树控制在 **train** 上的 Spearman 为正（第 8 层 0.167–0.170），而 dev 上为 0——探针确实把训练句的随机树背了下来，但这种记忆完全不迁移。这说明随机树目标只能靠逐词记忆解决，而表示里没有、也不可能提供这种随机结构的几何；反过来，真树的 selectivity 全层强正，说明模型表示提供的是真结构信息。

表 4 距离探针 selectivity (dev, gold UUAS – 随机树控制 UUAS)

层	BERT Δ	BERT 控制	RoBERTa Δ	RoBERTa 控制	MacBERT Δ	MacBERT 控制
0	0.279	0.096	0.276	0.099	0.280	0.099
2	0.349	0.094	0.349	0.101	0.359	0.096
4	0.440	0.098	0.460	0.097	0.461	0.097
6	0.454	0.098	0.482	0.096	0.473	0.093
8	0.491	0.096	0.489	0.098	0.492	0.097
10	0.437	0.097	0.442	0.097	0.442	0.096
12	0.387	0.100	0.386	0.099	0.379	0.096

(c) MDL：用比特数独立复现第 8 层。表 5 与图 3。

表 5 MDL：45 类依存关系标签的在线编码长度 (train, 越小越好)

层	BERT kbit	BERT 压缩率	BERT acc	RoBERTa kbit	RoBERTa 压缩率	RoBERTa acc	MacBERT kbit	MacBERT 压缩率	MacBERT acc
0	257.4	2.10	0.512	260.0	2.08	0.512	258.4	2.10	0.511
1	227.6	2.38	0.600	219.1	2.47	0.629	218.6	2.48	0.630
2	200.4	2.70	0.677	194.5	2.78	0.696	193.1	2.81	0.698
3	176.5	3.07	0.744	173.3	3.13	0.758	171.5	3.16	0.761
4	165.9	3.27	0.775	164.2	3.30	0.781	163.3	3.32	0.783
5	166.1	3.26	0.775	165.4	3.28	0.782	166.9	3.25	0.781
6	163.7	3.31	0.785	164.5	3.29	0.789	167.3	3.24	0.787
7	159.0	3.41	0.793	160.6	3.37	0.796	163.1	3.32	0.795
8	157.1	3.45	0.799	157.9	3.43	0.801	160.7	3.37	0.798
9	161.9	3.34	0.789	162.9	3.32	0.790	165.3	3.28	0.787
10	167.4	3.24	0.779	168.3	3.22	0.774	171.1	3.17	0.773
11	171.2	3.16	0.770	173.3	3.12	0.762	177.0	3.06	0.760
12	187.5	2.89	0.748	181.6	2.98	0.755	191.8	2.82	0.742

(均匀编码 541.6 kbit；压缩率 = 均匀码长 / 实际码长。)

从层 0 到层 8，压缩率由 2.10× 升到 3.45×，test 准确率由 0.512 升到 0.799；其间层 4→5 三个模型都出现一次极小回升（码长 +0.2 / +1.2 / +3.6 kbit，MacBERT 层 5→6 再 +0.4 kbit），不改整体趋势。从层 8 到层 12 则严格单调回升到 2.82–2.98×、准确率 0.742–0.755。**三个模型的最低码长都在第 8 层**，与探针峰值、selectivity 峰值三者重合。

MDL 的意义在于它换了一把尺子：探针指标衡量"能不能读出来"，MDL 把探针的复杂度也算进代价，衡量"信息是否被摆在了容易被廉价读出、且能泛化到新数据的位置"。二者独立地指向第 8 层，因而"探针容量过强造成的假阳性"这一质疑在此不成立。

4.4 小结：三条证据线的重合

证据线	度量	峰值层（三单语模型一致）	峰值处的值（BERT / RoBERTa / MacBERT）
结构探针（距离）	dev UUAS	层 8	0.587 / 0.587 / 0.589
结构探针（深度）	dev 深度 Spearman	层 7–8	0.799 / 0.803 / 0.804
控制任务 selectivity（深度）	ΔSpearman	层 8	0.071 / 0.072 / 0.113
控制任务 selectivity（距离）	ΔUUAS	层 7–8	0.491 / 0.489 / 0.492（首字）；0.519 / 0.512 / 0.519（均值）
MDL	编码比特数（最小）	层 8	157.1 / 157.9 / 160.7 kbit

四项稳健性检查的结论：

- **换训练目标**（LM / wwm / 纠错式）：峰值同层、形状同形，强度差 ≤2.6 个百分点（4.2a）。
- **换模型来源**（多语 mBERT）：峰值提前到层 7，强度略高（dev 0.604、test 0.619），selectivity 与 MDL 同向（4.2b）。
- **换词表示池化**（词首 → 词内均值）：UUAS 峰值层不变，绝对值上移约 3 个百分点，深度与距离两类 selectivity 同步上移（层 8 距离 ΔUUAS 0.489–0.492 → 0.512–0.519），控制分数不动（图 4(c)、4.2c）。
- **换目标结构**（真树 → 随机树 / 随机假深度）：真结构的优势在检验后依然存在，且在第 8 层最强（4.3）。

5 讨论

为什么峰值在第 8 层，而不是最高层？ 表示的使用方式提供了最自然的解释。第 9–12 层的指标（UUAS 从 0.587 掉到 0.48 左右，MDL 码长回升 20%）集体下降，而第 12 层恰好是离输出最近的层：预训练目标要求它把表示重新"摊开"成便于词表预测的形式，即向词汇—语义特化倾斜。句法作为中间层产物被保留下来，到顶层则服务于重构任务。这与英文模型上的层间分工一致：Tenney et al. (2019) 发现 BERT 的层序大致复现经典 NLP 流水线（词性 → 句法 → 命名实体 → 语义角色 → 共指），且性能在最后 1–2 层回落，说明句法这类中间步骤的信息集中于中层、到顶层让位于语义与重构。这一分工因此不是中文或某一训练策略特有的。

与跨语言证据的关系。 Chi et al. (2020) 在多语 mBERT 上发现中文是编码较差的语言之一——这是一个 mBERT 内部的跨语言相对比较。本文在同一流水下补跑了 mBERT，得到的是一个方向相反但并不冲突的结果：mBERT 在中文上的绝对可解码性（dev 峰值 0.604、test 0.619）**不低于**三个专门训练的中文单语模型（dev 0.587–0.589、test 0.596–0.605），峰值层还早一层。一个模型可以在自己的语言谱系里相对偏弱，同时仍然优于单语模型；我们的数据支持后一种读法。需要提醒的是，跨论文的绝对数值受树库版本、对齐方式与超参影响很大，因此本文所有比较都限定在同一条流水线、同一个树库、同一组超参内，不与文献中的数字直接对比。

方法论结论（对中文探针研究的普遍意义）。 低层负 selectivity 是最值得强调的一条：如果一项中文探针研究不做控制任务，它极可能在层 0–4 高估句法的可解码性，因为那里最容易读出的其实是字/词身份信息。我们的数据给出了这条警告的具体量级——层 0 的"表观句法可解码性"高达 0.375 UUAS，其中 0.096 属于任何随机结构都能达到的水平，而 selectivity 显示该层实际是负的。

局限。①单一 seed，未做多种子统计检验，故 1 个百分点量级的排序差异（4.2b 节的模型间对比）不应被当作结论；②单一树库（UD Chinese-GSDSimp，Wikipedia 语域，500 句 dev/test），绝对数值对树库与标注体系敏感；③探针范式只能证明信息“可解码”，不能证明模型在推理时“因果地使用”该信息；④未覆盖更大规模与生成式架构（Qwen 系列），因此“峰值在第 7–8 层”是否随规模漂移仍未知；⑤均值池化消融只覆盖三个单语模型，mBERT 未做池化对照；⑥评估把标点词计入依存图（3.5.4），因此本文 UUAS 的绝对值高于“排除标点”的常见口径，跨论文的数值不宜逐位比较（本文所有比较都在同一流水线内）。（“中文特有的首字池化会影响结论”这一顾虑已由 4.2c 的消融排除，不再列为局限。）

6 结论

本文在 UD Chinese-GSDSimp 上对四个同规模模型（三个中文单语 PLM 与多语 mBERT）施加了 Hewitt & Manning 结构探针、双向控制任务与 MDL 检验。三条独立证据线（可解码性、selectivity、编码长度）一致地把汉语句法几何的峰值定位在第 8 层（mBERT 为第 7 层）；该几何对预训练遮蔽策略高度鲁棒（峰值同层，全层差距 ≤ 2.6 个百分点），也不依赖中文特有的字 $\leftrightarrow$ 词池化方式（均值池化下峰值层与形状不变，绝对值上移约 3 个百分点）；低层的表面可解码性经控制检验后大部分归因于词形记忆，而中层的句法信号在检验下存活。多语 mBERT 在本流水线内并不弱于单语中文模型。这些结果支持一个温和但明确的结论：中文 PLM 确实自己的表示几何中编码了依存句法，这种编码的位置与强度主要由“层在计算中的角色”决定，而不由具体的预训练目标或语言覆盖范围决定。

下一步工作按优先级：①多种子重复，把 1 个百分点量级的模型差异变成可信的统计陈述；②在更大规模与生成式模型（Qwen 系列、Zh-Pythia 阶梯）上检验“峰值层随规模漂移”的假设；③把现象级探针（把/被字句、体标记）对接 CLiMP / SLING 中模型行为失败的类目，看“表示里有没有”与“行为上会不会”是否解离。

参考文献

（书目细节经 ACL Anthology / 出版社页面 / UD 官方仓库 / arXiv 等一手来源核实：首轮核对于 2026-10-02，RoBERTa 一条于 2026-10-04 补核；页码均取自一手页面元数据，无估算值。Hugging Face 模型卡在本机网络下不可达，模型出处改由官方 GitHub 仓库核实。）

- Devlin, Chang, Lee & Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019, 4171–4186. <https://aclanthology.org/N19-1423/> (bert-base-chinese 的配套论文)
- Liu, Ott, Goyal, Du, Joshi, Chen, Levy, Lewis, Zettlemoyer & Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019. <https://arxiv.org/abs/1907.11692> (hfl/chinese-roberta-wwm-ext 沿用的训练配方)
- Hewitt & Manning. A Structural Probe for Finding Syntax in Word Representations. NAACL-HLT 2019, 4129–4138. <https://aclanthology.org/N19-1419/>
- Hewitt & Liang. Designing and Interpreting Probes with Control Tasks. EMNLP-IJCNLP 2019, 2733–2743. <https://aclanthology.org/D19-1275/>
- Tenney, Das & Pavlick. BERT Rediscovered the Classical NLP Pipeline. ACL 2019, 4593–4601. <https://aclanthology.org/P19-1452/>
- Clark, Khandelwal, Levy & Manning. What Does BERT Look At? An Analysis of BERT's Attention. BlackboxNLP @ ACL 2019, 276–286. <https://aclanthology.org/W19-4828/>
- Voita & Titov. Information-Theoretic Probing with Minimum Description Length. EMNLP 2020, 183–196. <https://aclanthology.org/2020.emnlp-main.14/>

- Pimentel, Cotterell, Teufel & Blunsom. Information-Theoretic Probing for Linguistic Structure. ACL 2020, 4609–4622. <https://aclanthology.org/2020.acl-main.420/>
- Pimentel, Saphra, Teufel, Cotterell & Blunsom. Pareto Probing: Trading Off Accuracy for Complexity. EMNLP 2020, 3138–3153. [arXiv:2010.02180](https://arxiv.org/abs/2010.02180)
- Chi, Hewitt & Manning. Finding Universal Grammatical Relations in Multilingual BERT. ACL 2020, 5564–5577. <https://aclanthology.org/2020.acl-main.493/>
- Kulmizev, Ravishankar, Abdou & Nivre. Do Neural Language Models Show Preferences for Syntactic Formalisms? ACL 2020, 4077–4091. <https://aclanthology.org/2020.acl-main.375/>
- Xiang, Yang, Li, Warstadt & Kann. CLiMP: A Benchmark for Chinese Language Model Evaluation. EACL 2021, 2784–2790. <https://aclanthology.org/2021.eacl-main.242/>
- Song, Krishna, Bhatt & Iyyer. SLING: Sino Linguistic Evaluation of Large Language Models. EMNLP 2022, 4606–4634. <https://aclanthology.org/2022.emnlp-main.305/>
- Liu, Shen, Zhu, Xu, Qian, Song, Zhang, Tang, Zhang, Yang, Wang & Hu. A Systematic Assessment of Language Models with Linguistic Minimal Pairs in Chinese (ZhoBLiMP 数据集) . TACL, Vol. 14, 2026, 755–771. <https://aclanthology.org/2026.tacl-1.34/> ; 预印本 [arXiv:2411.06096](https://arxiv.org/abs/2411.06096)
- Zheng & Liu. What does Chinese BERT learn about syntactic knowledge? PeerJ Computer Science 9:e1478, 2023. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10403162/>
- Cui, Che, Liu, Qin & Yang. Pre-Training with Whole Word Masking for Chinese BERT. IEEE/ACM TASLP, Vol. 29, 2021, 3504–3514. <https://doi.org/10.1109/TASLP.2021.3124365> (hf1/chinese-roberta-wwm-ext 的出处)
- Cui, Che, Liu, Qin, Wang & Hu. Revisiting Pre-Trained Models for Chinese Natural Language Processing (MacBERT) . Findings of EMNLP 2020, 657–668. <https://aclanthology.org/2020.findings-emnlp.58/> (hf1/chinese-macbert-base 的出处)
- Rogers, Kovaleva & Rumshisky. A Primer in BERTology: What We Know About How BERT Works. TACL, Vol. 8, 2020, 842–866. <https://aclanthology.org/2020.tacl-1.54/>
- Nivre et al. Universal Dependencies v1: A Multilingual Treebank Collection. LREC 2016, 1659–1666. <https://aclanthology.org/L16-1262/>
- UD Chinese-GSDSimp 树库无独立论文，随 UD 数据发布引用（UD v2.5 起纳入；贡献者 Peng Qi、Koichi Yasuoka；许可 CC BY-SA 4.0）。
https://universaldependencies.org/treebanks/zh_gsdsimp/index.html

图表清单

- 图 1：四模型 × 13 层 UUAS / 距离 Spearman 曲线（含随机树基线） — [figures/fig1_syntax_geometry.png](#)
- 图 2：selectivity 随层变化（深度 / 距离，四模型） — [figures/fig2_selectivity.png](#)
- 图 3：MDL 压缩率与依存关系分类准确率（四模型） — [figures/fig3_md1.png](#)
- 图 4：均值池化消融（三模型，首字实线 vs 均值虚线）——(a) dev UUAS、(b) MDL 压缩率、(c) 距离探针 selectivity — [figures/fig4_pooling_ablation.png](#)
- 表 1：dev 逐层全量指标（三单语模型）；表 2：test 逐层 UUAS/距离 Spearman；表 3：深度 selectivity；表 4：距离 selectivity；表 5：MDL；表 6–7：mBERT 逐层指标与四模型峰值对比；表 8：均值池化消融（全部数字经 `src/summarize.py` 从 `results/*.json` 自动生成，见 `docs/results-tables.md`）

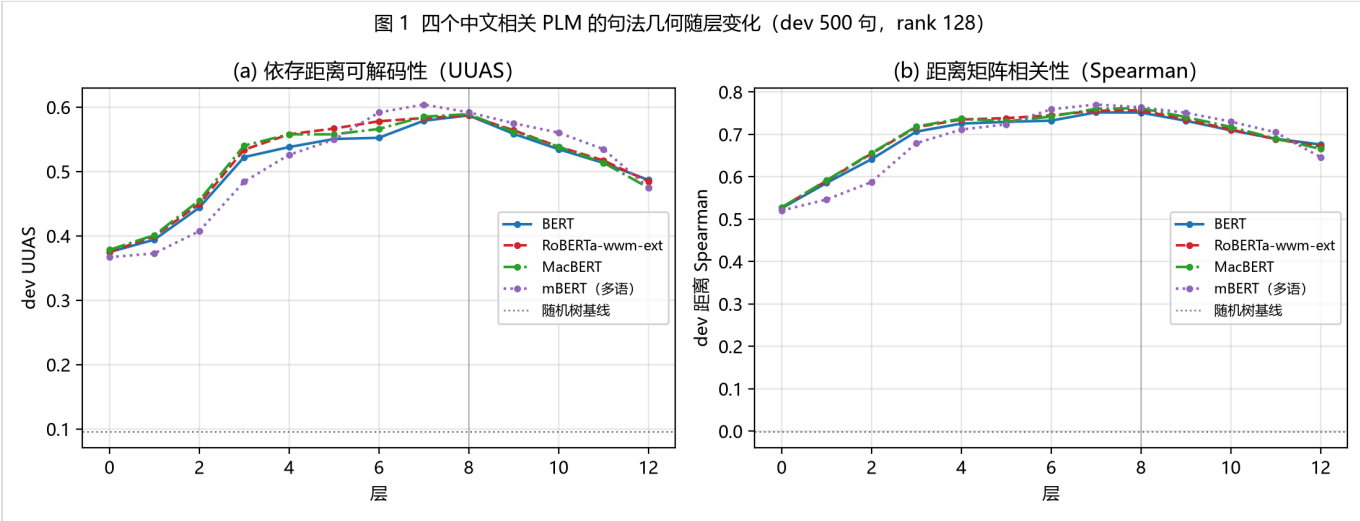


图 1 四模型 × 13 层 UUAS / 距离 Spearman 曲线 (含随机树基线)

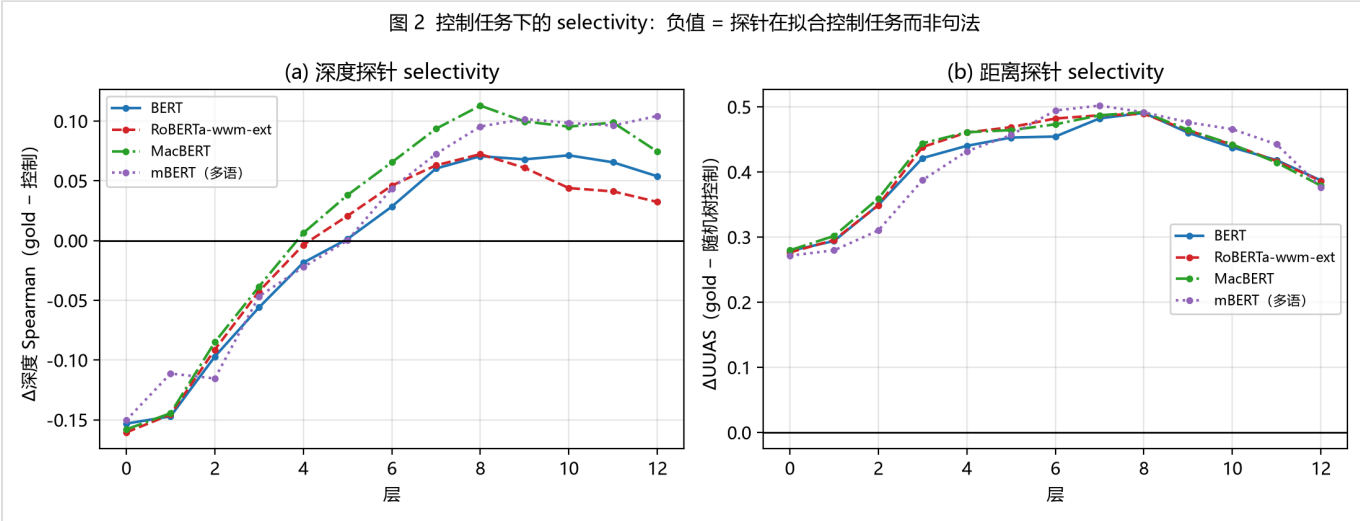


图 2 selectivity 随层变化 (深度 / 距离, 四模型)

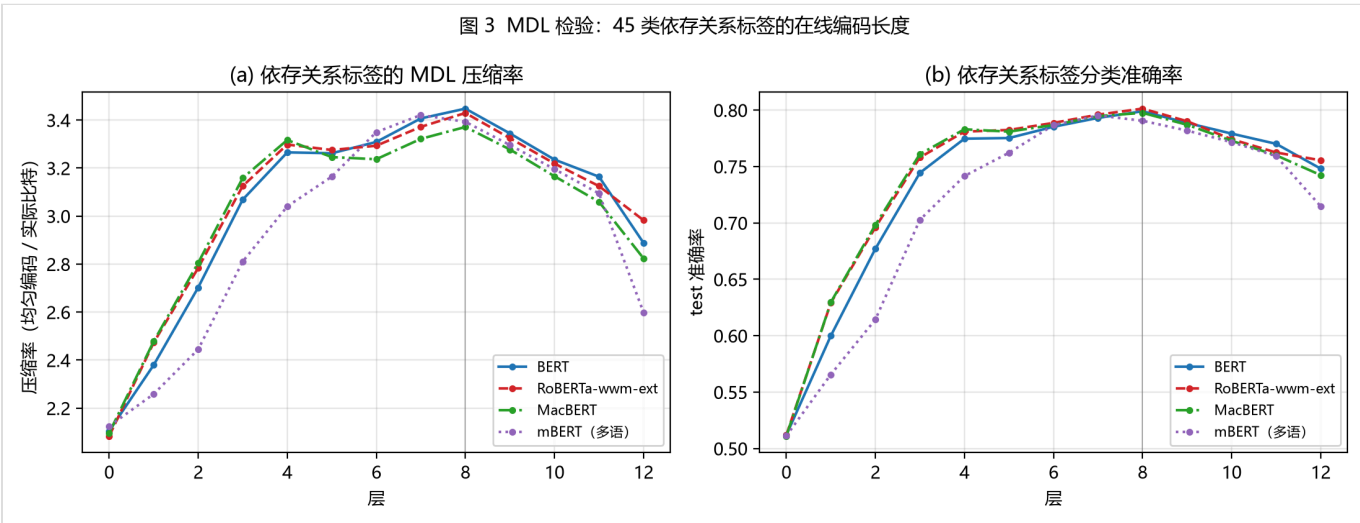


图 3 MDL 压缩率与依存关系分类准确率 (四模型)

图 4 均值池化消融：峰值层不变，绝对值整体上移

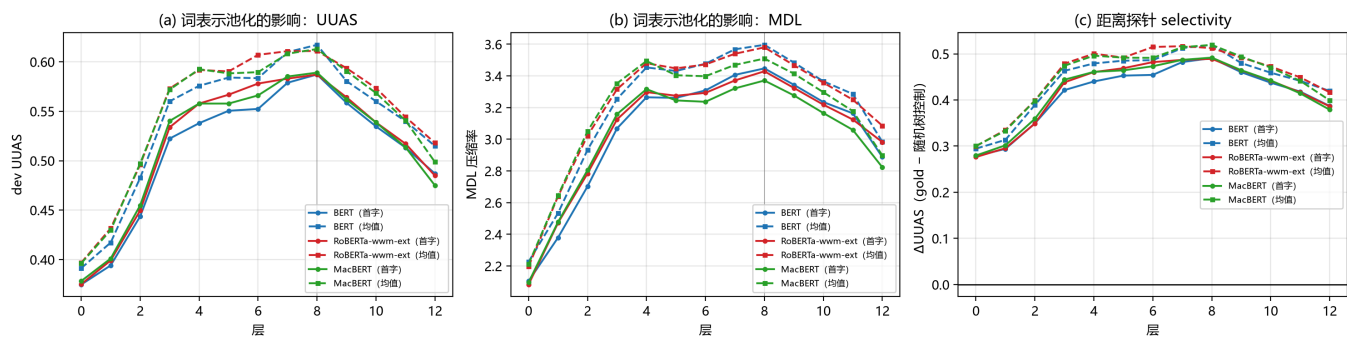


图 4 均值池化消融 (三模型, 首字实线 vs 均值虚线) : (a) dev UUAS、(b) MDL 压缩率、(c) 距离探针 selectivity